PES University, Bengaluru

Department of Computer Science and Engineering

MTech in Data Science & Artificial Intelligence

RL for LLMs: A Systematic GRPO Scaling Study

Capstone Project Report

Group 6

Academic Year 2025–2026

Chapter 2

Foundation

2.1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities across a broad spectrum of natural-language tasks, yet their raw pre-trained behaviour often diverges from the nuanced requirements of downstream applications. Bridging this gap—aligning model outputs with human intent, factual correctness, and structured reasoning—has become one of the central challenges of modern NLP research. The present chapter lays the theoretical and empirical groundwork upon which our scaling study is built, surveying the key ideas, algorithms, and benchmarks that collectively motivate and inform our experimental design.

Reinforcement Learning from Human Feedback (RLHF). The RLHF pipeline, popularised by InstructGPT (Ouyang et al., 2022), established a three-stage recipe—supervised fine-tuning, reward-model training, and reinforcement-learning-based policy optimisation—that transformed how practitioners align LLMs with human preferences. We begin in Section 2.2 by reviewing this pipeline in detail, since every subsequent algorithmic advance discussed in this chapter can be understood as an attempt to simplify, stabilise, or scale one or more of its stages.

Policy optimisation: from PPO to DPO to GRPO. Proximal Policy Optimisation (PPO; Schulman et al., 2017) served as the workhorse optimiser within early RLHF systems, but its reliance on a separately trained value network and careful hyperparameter tuning introduced significant computational overhead. Direct Preference Optimisation (DPO; Rafailov et al., 2023) later showed that the reward-modelling step could be bypassed entirely by re-parameterising the Bradley–Terry preference objective in terms of the policy itself. Most recently, Group Relative Policy Optimisation (GRPO; Shao et al., 2024) proposed an elegant middle ground: it retains the explicit reward signal of PPO but eliminates the value function by computing advantages relative

to a group of sampled completions, thereby reducing memory requirements and simplifying the training loop. The DeepSeek-R1 effort (DeepSeek-AI, 2025) subsequently demonstrated that GRPO can elicit sophisticated multi-step reasoning in large-scale models. Sections 2.3–2.5 trace this algorithmic lineage, formalise each objective, and analyse the trade-offs that make GRPO an attractive choice for resource-constrained scaling experiments.

Parameter-efficient fine-tuning. Full fine-tuning of models with billions of parameters is prohibitively expensive for most research groups. Low-Rank Adaptation (LoRA; Hu et al., 2022) offers a compelling alternative by injecting trainable low-rank matrices into frozen pre-trained weights, dramatically reducing the number of trainable parameters while preserving expressive power. In Section 2.7, we review LoRA and its variants, discussing how they interact with reinforcement learning objectives and why they are essential enablers of the scaling experiments presented in later chapters.

Mathematical reasoning benchmarks. Rigorous evaluation requires benchmarks with unambiguous ground-truth answers. We focus on two widely adopted mathematical reasoning datasets: GSM8K (Cobbe et al., 2021), a collection of grade-school math word problems, and MATH (Hendrycks et al., 2021c), a more challenging suite spanning algebra, number theory, geometry, and beyond. Together with chain-of-thought prompting strategies (Wei et al., 2022), these benchmarks provide a controlled test-bed for measuring whether reinforcement-learning-based fine-tuning genuinely improves a model's capacity for step-by-step reasoning rather than superficial pattern matching. Section 2.8 describes these datasets, their evaluation protocols, and the metrics we adopt.

Related scaling studies and infrastructure. Finally, Sections 2.9–2.11 survey prior work that has investigated how RLHF and its variants behave as a function of model size, dataset scale, and compute budget, along with the infrastructure that makes such experiments feasible. These studies provide important baselines and methodological precedents for our own systematic exploration of GRPO scaling behaviour.

Taken together, the topics covered in this chapter establish the conceptual vocabulary, algorithmic toolkit, and evaluation methodology required to appreciate the experimental contributions of the subsequent chapters. By grounding our study in this foundation, we aim to make the design choices, trade-offs, and results of our GRPO scaling experiments both transparent and reproducible.

2.2 Reinforcement Learning from Human Feedback

The core challenge in deploying large language models is the misalignment between the pretraining objective—predicting the next token on internet text—and the downstream goal of generating outputs that are helpful, honest, and harmless. Reinforcement Learning from Human Feedback (RLHF) emerged as the dominant paradigm for bridging this gap, establishing a systematic pipeline that transforms raw human preference judgments into a training signal capable of steering model behavior. This section traces the intellectual lineage of RLHF, formalizes its three-stage pipeline, and examines the limitations that have motivated subsequent alternatives.

2.2.1 Origins and Early Work

The idea of learning reward functions from human preferences, rather than hand-engineering them, predates the modern LLM era. **?** proposed a framework for *deep reinforcement learning from human preferences*, demonstrating that a reward model trained on pairwise comparisons of trajectory segments could guide an RL agent to solve complex tasks—including Atari games and simulated robot locomotion—without access to the ground-truth reward function. Critically, their approach required human feedback on less than one percent of the agent's total interactions with the environment, making human oversight practically feasible. This work established two foundational principles: (i) pairwise comparisons are a more reliable and cognitively simpler annotation format than absolute scalar ratings, and (ii) a learned reward model can serve as a proxy objective that decouples the expensive human evaluation from the iterative policy optimization loop.

The transition from simulated environments to natural language was pioneered by **?**, who applied reward learning to fine-tune language models on four tasks: stylistic text continuation and abstractive summarization. Using a pretrained language model as the initial policy, they trained a reward model on approximately 60,000 human comparisons and optimized the policy with Proximal Policy Optimization (PPO) Schulman et al. (2017). A key technical contribution was the introduction of a Kullback–Leibler (KL) divergence penalty between the learned policy and the original supervised model, which prevented the policy from deviating too far from the pretrained distribution—a regularization mechanism that would become standard in all subsequent RLHF work.

? scaled this approach substantially, applying it to the Reddit TL;DR summarization task with models up to 6.7 billion parameters. They collected a high-quality dataset of 64,832 human comparisons and demonstrated that policies trained with human feedback significantly outperformed supervised learning baselines—even those with ten times more parameters. This work also provided early empirical evidence of *reward over-optimization*: as the policy was optimized more aggressively against the re-

ward model, true human preference scores initially improved but eventually degraded, revealing that the reward model's predictions became anti-correlated with genuine quality under heavy optimization.

2.2.2 The Three-Stage RLHF Pipeline

The work of Ouyang et al. (2022) consolidated and scaled the preceding research into the now-canonical three-stage RLHF pipeline, applying it to align GPT-3 across a broad distribution of real-world user instructions. The resulting system, **InstructGPT**, became the methodological foundation for ChatGPT and established RLHF as the dominant post-training paradigm for large language models.

Stage 1: Supervised Fine-Tuning (SFT). The pipeline begins with supervised fine-tuning of a pretrained language model on a dataset of high-quality demonstrations. In the InstructGPT setting, a team of 40 human contractors wrote demonstrations of desired model behavior for a diverse set of prompts. A pretrained GPT-3 model was then fine-tuned on approximately 13,000 such demonstration prompts using standard maximum likelihood estimation. The resulting SFT model serves a dual purpose: it provides a strong initialization for the RL policy and establishes a reference distribution against which policy divergence is subsequently penalized.

Stage 2: Reward Model Training. The second stage trains a scalar reward model that serves as a proxy for human judgment. Starting from the SFT model with its final unembedding layer replaced by a randomly initialized linear projection head, the reward model is trained to predict which of two candidate completions a human annotator would prefer for a given prompt. Ouyang et al. (2022) employed a 6B parameter reward model, finding that 175B reward models exhibited training instability.

The reward model is trained under the **Bradley–Terry** pairwise comparison framework. Given a prompt x and a pair of completions where y_w is preferred over y_l by a human annotator, the probability that y_w is preferred is modeled as:

$$P(y_w \succ y_l \mid x) = \sigma\big(r_\theta(x, y_w) - r_\theta(x, y_l)\big), \tag{2.1}$$

where $r_\theta(x, y)$ is the scalar reward assigned by the model with parameters θ, and $\sigma(\cdot)$ denotes the logistic sigmoid function. The corresponding training loss is:

$$\mathcal{L}_{\mathrm{RM}}(\theta) = -\frac{1}{\binom{K}{2}} \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\Big[\log \sigma\big(r_\theta(x, y_w) - r_\theta(x, y_l)\big)\Big], \tag{2.2}$$

where K is the number of candidate responses ranked per prompt.

Stage 3: Reinforcement Learning with PPO. In the final stage, the SFT model is fine-tuned as an RL policy to maximize the learned reward signal using PPO Schulman et al. (2017). To prevent reward hacking, a per-token KL divergence penalty constrains the RL policy to remain close to the reference SFT distribution:

$$\mathcal{J}(\phi) = \mathbb{E}_{(x,y)\sim\pi_\phi^{\mathrm{RL}}}\Big[r_\theta(x,y) - \beta\,\mathrm{KL}\big(\pi_\phi^{\mathrm{RL}}(\cdot \mid x) \,\|\, \pi^{\mathrm{SFT}}(\cdot \mid x)\big)\Big], \tag{2.3}$$

where β controls the strength of the KL penalty.

2.2.3 InstructGPT: Results and Significance

The results of Ouyang et al. (2022) provided compelling evidence for the effectiveness of RLHF at scale. The headline finding was striking: *the 1.3B parameter InstructGPT model was preferred by human evaluators over the 175B parameter GPT-3*, despite having more than 100× fewer parameters. The 175B InstructGPT model was preferred over the 175B GPT-3 baseline 85% of the time. On TruthfulQA, InstructGPT generated truthful and informative answers approximately twice as often as GPT-3, and hallucination rates on closed-domain tasks dropped from 41% to 21%.

2.2.4 Limitations of the Standard RLHF Pipeline

Despite its empirical success, the standard RLHF pipeline introduces several significant challenges:

Computational Complexity. The full RLHF pipeline requires maintaining up to four models simultaneously in GPU memory: (i) the policy model, (ii) the reference model, (iii) the reward model, and (iv) the value/critic model. For large-scale models, this quadrupling of memory requirements imposes severe infrastructure constraints.

Training Instability. PPO-based RLHF training is notoriously sensitive to hyperparameters, including the KL penalty coefficient β, the PPO clipping ratio, learning rates, and the number of PPO epochs per batch.

Reward Hacking and Over-Optimization. As the policy is optimized against the learned proxy reward, it inevitably discovers outputs that achieve high reward model scores without corresponding improvements in genuine quality—a failure mode known as *reward hacking* or *Goodhart's Law* applied to learned objectives.

Human Annotation Bottleneck. The pipeline depends critically on the quality, consistency, and scale of human preference annotations, creating a bottleneck that limits iteration speed.

These limitations—particularly the computational overhead and the fragility of PPO-based optimization—have driven the development of alternative approaches discussed in the following sections.

2.3 Proximal Policy Optimization for Language Models

Proximal Policy Optimization (PPO), introduced by Schulman et al. (2017), has become the dominant reinforcement learning algorithm for aligning large language models with human preferences. This section presents the PPO algorithm, describes its adaptation to language modeling, and discusses the practical challenges that motivate the search for simpler alternatives.

2.3.1 The PPO Algorithm

Policy gradient methods optimize a parameterized policy π_θ by ascending the gradient of an expected return objective. Let $r_t(\theta)$ denote the probability ratio between the current and old policies:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}. \tag{2.4}$$

The clipped surrogate objective is:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min\Big(r_t(\theta)\, \hat{A}_t,\ \ \text{clip}\big(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon\big)\, \hat{A}_t \Big) \right], \tag{2.5}$$

where ϵ is a hyperparameter (typically $\epsilon = 0.2$) controlling the width of the trust region.

PPO employs an **actor-critic architecture** with the full objective:

$$L^{\text{CLIP+VF+S}}(\theta) = \hat{\mathbb{E}}_t \Big[L_t^{\text{CLIP}}(\theta) - c_1\, L_t^{\text{VF}}(\theta) + c_2\, S[\pi_\theta](s_t) \Big], \tag{2.6}$$

where $L_t^{\text{VF}}(\theta) = \big(V_\theta(s_t) - \hat{R}_t\big)^2$ is the value function loss and $S[\pi_\theta](s_t)$ is the entropy bonus.

2.3.2 Generalized Advantage Estimation

Generalized Advantage Estimation (GAE) provides a principled mechanism for interpolating between high-bias and low-variance advantage estimates. Define the TD residual as $\delta_t = r_t + \gamma\, V(s_{t+1}) - V(s_t)$. The GAE advantage estimator is:

$$\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l\, \delta_{t+l}, \tag{2.7}$$

where $\lambda \in [0, 1]$ controls the bias-variance trade-off.

2.3.3 Recasting PPO for Language Generation

Applying PPO to language model alignment requires reinterpreting the RL formalism:

- **State** (s_t): the prompt concatenated with all tokens generated so far.
- **Action** (a_t): the next token y_t sampled from the LLM's output distribution.
- **Policy** (π_θ): the language model itself.
- **Reward**: a scalar signal from a learned reward model $r_\phi(x, y)$ upon completion of the full response, making credit assignment across the sequence a central challenge.

2.3.4 The Four-Model Architecture

A distinctive characteristic of PPO-based RLHF is its requirement for **four separate neural network models** during training Ouyang et al. (2022); Zheng et al. (2023):

1. **Policy model** π_θ: the language model being optimized.
2. **Reference model** π^{SFT}: a frozen copy for computing the KL penalty.
3. **Reward model** r_ϕ: scores generated completions.
4. **Value model (critic)** V_ψ: estimates expected cumulative reward for GAE.

For a 70B-parameter LLM, the effective memory footprint is roughly four times that of standard supervised fine-tuning.

2.3.5 Challenges of PPO for Large Language Models

Computational cost. The four-model architecture and on-policy sampling impose enormous memory and compute burdens.

Hyperparameter sensitivity. Zheng et al. (2023) conducted an extensive empirical investigation, identifying *policy constraints* as the single most critical factor for stable training.

Reward model dependence. The quality of the reward model places an upper bound on the quality of the final policy.

Implementation complexity. Subtle details such as advantage whitening, reward clipping, and global gradient norm clipping all materially affect training stability Zheng et al. (2023).

These challenges collectively explain why PPO-based RLHF has been described as a "puzzle" to stabilize at scale and provide the direct motivation for the development of simpler RL algorithms for LLMs.

2.4 Direct Preference Optimization

Direct Preference Optimization (DPO) Rafailov et al. (2023) eliminates the reward-modelling and reinforcement-learning stages entirely by showing that the optimal policy under the KL-constrained RLHF objective admits a closed-form solution.

Closed-form optimal policy. The KL-constrained RLHF objective seeks:

$$\max_{\pi} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi(\cdot|x)}\big[r(x,y)\big] - \beta\, D_{\mathrm{KL}}\big(\pi(\cdot|x) \,\|\, \pi_{\mathrm{ref}}(\cdot|x)\big). \tag{2.8}$$

The unique optimum is:

$$\pi^*(y|x) = \frac{1}{Z(x)}\, \pi_{\mathrm{ref}}(y|x)\, \exp\!\Big(\frac{1}{\beta}\, r(x,y)\Big), \tag{2.9}$$

where $Z(x)$ is the partition function.

Reward reparameterization. Rearranging yields the reward in terms of the optimal policy:

$$r(x,y) = \beta\, \log \frac{\pi^*(y|x)}{\pi_{\mathrm{ref}}(y|x)} + \beta\, \log Z(x). \tag{2.10}$$

The DPO loss. Substituting into the Bradley–Terry preference model **?**, the partition functions cancel, giving:

$$\boxed{\mathcal{L}_{\mathrm{DPO}}(\theta) = -\,\mathbb{E}_{(x,\,y_w,\,y_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta\left(\log \frac{\pi_\theta(y_w|x)}{\pi_{\mathrm{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\mathrm{ref}}(y_l|x)}\right)\right)\right].} \tag{2.11}$$

Practical advantages. DPO enjoys several benefits: (1) no separate reward model or RL optimizer required; (2) 40–60% less GPU memory than PPO; (3) stable cross-entropy-style training.

Limitations. DPO has well-documented shortcomings: (1) dependence on offline, static preference data leading to distribution shift; (2) no online exploration; (3) likelihood displacement; (4) weak credit assignment for long-horizon reasoning tasks.

Why DPO is insufficient for mathematical reasoning. Mathematical reasoning is *verifiable*—a derivation either reaches the correct answer or not—and this binary outcome signal is far richer and cheaper to obtain than human preference annotations. DPO cannot exploit this signal because it requires pre-collected pairwise preferences. Furthermore, the credit-assignment problem is severe: a single error in one step renders the entire completion incorrect, but DPO's completion-level loss provides no mechanism to localise fault. These shortcomings motivate GRPO, introduced next.

2.5 Group Relative Policy Optimization

Group Relative Policy Optimization (GRPO), introduced by Shao et al. (2024) in the DeepSeekMath project, charts an elegant middle course: it retains the on-policy sampling and explicit reward signal of PPO while discarding the value network that constitutes its principal computational bottleneck.

Motivation: the cost of the critic. The standard PPO setup loads four full-sized models into GPU memory. GRPO eliminates V_ψ entirely by *computing* a baseline on the fly through group sampling and reward normalization, reducing the number of trained models from four to three.

The GRPO algorithm. GRPO proceeds in three steps:

Step 1: Group sampling. For each prompt q, the current policy generates a group of G independent completions:

$$\{o_1, o_2, \ldots, o_G\} \sim \pi_{\theta_{\text{old}}}(\cdot \mid q). \tag{2.12}$$

Step 2: Group-relative advantage estimation. A reward function scores each completion, yielding $\mathbf{r} = \{r_1, \ldots, r_G\}$. Advantages are computed by z-score normalization within the group:

$$\boxed{\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}, \qquad i = 1, \ldots, G, \quad t = 1, \ldots, |o_i|.} \tag{2.13}$$

Step 3: Clipped policy update with KL regularization.

$$\boxed{\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{q\sim\mathcal{D},\, \{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{G}\sum_{i=1}^{G} \frac{1}{|o_i|}\sum_{t=1}^{|o_i|} \left(\min\Big(\rho_{i,t}\,\hat{A}_{i,t},\ \text{clip}\big(\rho_{i,t},\ 1-\varepsilon,\ 1+\varepsilon\big)\,\hat{A}_{i,t}\Big) - \beta\, D_{\text{KL}}^{(i,t)} \right) \right]} \tag{2.14}$$

Algorithm 1: Group Relative Policy Optimization (GRPO)

Input : Initial policy $\pi_{\theta_{\text{init}}}$;
reward model r_φ;
task prompts $\mathcal{D}$;
hyperparameters $\varepsilon, \beta, G, \mu$
Output: Optimized policy π_θ

```
π_θ ← π_θ_init
for iteration = 1, ..., I do
    π_ref ← π_θ                                          // Sync reference model
    for step = 1, ..., M do
        Sample a mini-batch D_b ⊂ D
        π_θ_old ← π_θ                                    // Snapshot policy
        foreach prompt q ∈ D_b do
            Sample G outputs {o_i}_{i=1}^G ~ π_θ_old(· | q)      // Step 1
            Compute rewards {r_i}_{i=1}^G using r_φ
            Â_{i,t} ← (r_i − mean(r)) / std(r)   ∀ i, t          // Step 2
        end
        for GRPO sub-iteration = 1, ..., μ do
            Update π_θ by maximizing J_GRPO(θ)   (Eq. 2.14)      // Step 3
        end
    end
end
return π_θ
```

where the per-token probability ratio is $\rho_{i,t} = \pi_\theta(o_{i,t} \mid q, o_{i,<t}) / \pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})$, and the per-token KL divergence uses the unbiased estimator:

$$D_{\text{KL}}^{(i,t)} = \frac{\pi_{\text{ref}}(o_{i,t} \mid q,\, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q,\, o_{i,<t})} - \log \frac{\pi_{\text{ref}}(o_{i,t} \mid q,\, o_{i,<t})}{\pi_\theta(o_{i,t} \mid q,\, o_{i,<t})} - 1. \tag{2.15}$$

Comparison with PPO.

Reward design for mathematical reasoning. Shao et al. (2024) exploit the verifiability of mathematics through rule-based reward functions:

- **Accuracy reward:** $r_{\text{acc}}(q, o) = \mathbf{1}[\text{extract}(o) = a^*]$, where a^* is the ground-truth answer.
- **Format reward:** A small bonus for structured chain-of-thought output (e.g., enclosing the final answer in `\boxed{}`).

The combination of verifiable binary rewards and group-relative normalization creates a self-regulating learning signal: when the model is weak and most completions

Table 2.1: Comparison of PPO and GRPO for LLM reinforcement learning.

Aspect	**PPO**	**GRPO**
Models in memory	Policy, reference, reward, value (4)	Policy, reference, reward (3)
Advantage estimation	GAE via learned V_ψ	Z-score within sampled group
KL regularization	Penalty added to reward	Penalty added directly to loss
Per-token value network	Required	Not needed
Sampling strategy	Single rollout per prompt	G rollouts per prompt
Hyperparameters	$\varepsilon, \gamma, \lambda, \beta$, value LR	ε, β, G

are wrong, the few correct ones receive very large positive advantages; as the model improves, advantage magnitudes naturally shrink.

DeepSeekMath results. On the competition-level MATH benchmark, DeepSeekMath-RL 7B achieved **51.7%** top-1 accuracy—competitive with Gemini-Ultra (53.2%) and GPT-4 (52.9%). With self-consistency over 64 samples, accuracy rose to **60.9%**. On GSM8K, GRPO improved accuracy from 82.9% to 88.2%, with gains transferring to out-of-domain benchmarks.

2.6 DeepSeek-R1 and Emergent Reasoning

DeepSeek-R1 DeepSeek-AI (2025) provides a compelling demonstration that reasoning abilities can emerge from reinforcement learning alone.

2.6.1 DeepSeek-R1-Zero: Pure RL without Supervised Fine-Tuning

DeepSeek-R1-Zero is trained by applying GRPO directly to DeepSeek-V3-Base DeepSeek-AI (2024), a 671-billion-parameter Mixture-of-Experts architecture, without any SFT stage. The reward design is entirely rule-based: accuracy rewards (exact match on final answer or test case execution) and format rewards (`<think>...</think>` tags). No neural reward model is employed.

On AIME 2024, pass@1 accuracy increases from an initial 15.6% to 71.0% over the course of training. With consensus decoding (cons@64), the score reaches 86.7%. On MATH-500, DeepSeek-R1-Zero achieves 95.9%.

2.6.2 Emergent Behaviors

Spontaneous chain-of-thought. The model naturally learns to allocate more "thinking time," with average response length increasing steadily throughout training.

Self-reflection and verification. The model develops the capacity to recognize errors and backtrack, spontaneously producing phrases such as "Wait, let me reconsider."

The "aha moment." An intermediate checkpoint begins producing reflections like: "*Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step.*" The authors describe this as "an aha moment for us, allowing us to witness the power and beauty of reinforcement learning."

2.6.3 The DeepSeek-R1 Multi-Stage Pipeline

To address R1-Zero's limitations (poor readability, language mixing), DeepSeek-R1 uses a four-stage pipeline:

Stage 1: Cold-start SFT. Fine-tuning on curated long chain-of-thought examples.

Stage 2: Reasoning-focused RL. GRPO with accuracy, format, and language-consistency rewards:

$$R_{\text{language}} = \frac{\text{Num}(\text{Words}_{\text{target}})}{\text{Num}(\text{Words}_{\text{total}})}. \tag{2.16}$$

Stage 3: Rejection sampling and SFT. Generating ∼800K samples via rejection sampling, combined with non-reasoning data.

Stage 4: RL across all scenarios. Final RL with diverse reward signals spanning reasoning, helpfulness, and safety.

2.6.4 Benchmark Results

Table 2.2: Benchmark comparison of DeepSeek models and OpenAI o1-1217. All figures are pass@1 (%).

Benchmark	**R1-Zero**	**R1**	**o1-1217**
AIME 2024 (pass@1)	77.9	79.8	79.2
MATH-500 (pass@1)	95.9	97.3	96.4
GPQA Diamond (pass@1)	75.8	71.5	75.7
Codeforces (percentile)	80.4	96.3	96.6
LiveCodeBench (pass@1)	50.0	65.9	63.4

2.6.5 Distillation to Smaller Models

DeepSeek-R1's reasoning capabilities transfer effectively to smaller models (1.5B–70B) through knowledge distillation. Critically, **distillation outperforms direct RL on smaller models**: DeepSeek-R1-Distill-Qwen-32B achieves 72.6% on AIME 2024 vs. 47.0% for Qwen2.5-32B-Zero (trained with large-scale RL from scratch). This suggests that below a certain capacity threshold, distilling reasoning patterns from a more powerful teacher is both more effective and more computationally efficient than RL from scratch.

2.7 Parameter-Efficient Fine-Tuning

The preceding sections have established that RL—and GRPO in particular—can substantially improve LLM reasoning capabilities. However, these algorithms presuppose the ability to *fine-tune* the underlying language model. For a model with N parameters, full fine-tuning with Adam requires roughly $16N$ bytes of GPU memory. For a 7B model, this alone exceeds 100 GB. The problem is compounded in RL settings where GRPO requires maintaining a reference policy alongside the active policy.

2.7.1 Low-Rank Adaptation (LoRA)

Low-Rank Adaptation (Hu et al., 2022) freezes $\mathbf{W}_0$ and injects a trainable low-rank decomposition:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{B}\mathbf{A}, \tag{2.17}$$

where $\mathbf{B} \in \mathbb{R}^{d\times r}$, $\mathbf{A} \in \mathbb{R}^{r\times k}$, and $r \ll \min(d, k)$. The forward pass is:

$$\mathbf{h} = \mathbf{W}_0\mathbf{x} + \frac{\alpha}{r}\mathbf{B}\mathbf{A}\mathbf{x}, \tag{2.18}$$

where α is a scaling hyperparameter. Initialization: $\mathbf{A} \sim \mathcal{N}(0, \sigma^2)$, $\mathbf{B} = \mathbf{0}$, ensuring $\mathbf{B}\mathbf{A} = \mathbf{0}$ at the start. At inference, the low-rank matrices merge into the base weight: $\mathbf{W}_{\text{merged}} = \mathbf{W}_0 + \mathbf{B}\mathbf{A}$, introducing *zero additional latency.*

Rank selection. Common choices include $r \in \{8, 16, 32, 64\}$. Even $r = 4$ can match full fine-tuning on many benchmarks.

Target modules. Adapting query ($\mathbf{W}_Q$) and value ($\mathbf{W}_V$) projections yields the best trade-off. For a 7B model with $r = 16$, LoRA trains ∼0.1% of total parameters.

Table 2.3: Comparison of parameter-efficient fine-tuning methods.

Method	Trainable params	Extra inference cost	Merges into bas
Full fine-tuning	100%	None	N/A
Adapters (**?**)	∼3–5%	Yes	No
Prefix tuning (**?**)	∼0.1–1%	Yes	No
Prompt tuning (**?**)	<0.01%	Minimal	No
LoRA (Hu et al., 2022)	∼0.1%	None	Yes
QLoRA (Dettmers et al., 2023)	∼0.1%	None	Yes

2.7.2 QLoRA: Quantised Low-Rank Adaptation

Dettmers et al. (2023) introduced QLoRA with three innovations:

1. **4-bit NormalFloat (NF4) quantization**: information-theoretically optimal for normally distributed weights.
2. **Double quantization**: reduces memory from quantization constants.
3. **Paged optimizers**: manages memory spikes via unified memory.

Together, these enable fine-tuning a 65B model on a single 48 GB GPU.

2.7.3 Other Parameter-Efficient Methods

2.7.4 LoRA for Reinforcement Learning Fine-Tuning

The combination of LoRA with GRPO confers several structural advantages:

Memory budget reallocation. LoRA frees GPU memory that can be reallocated to larger group sizes G and batch sizes.

Stable reference policy. The reference policy is simply the frozen base model $\mathbf{W}_0$—no separate copy needed. The reference output can be obtained by performing a forward pass with LoRA adapters disabled.

Regularization effect. The low-rank constraint acts as implicit regularization on the policy update, complementing GRPO's explicit KL penalty.

Efficient checkpointing. Each checkpoint consists only of the small $\mathbf{B}$ and $\mathbf{A}$ matrices (megabytes rather than gigabytes), facilitating rapid experimentation.

2.8 Evaluation Benchmarks

2.8.1 GSM8K

The Grade School Math 8K (GSM8K) benchmark Cobbe et al. (2021) comprises 8,500 grade-school-level math word problems (7,473 train, 1,319 test). Each problem requires 2–8 steps of elementary arithmetic reasoning. Evaluation is via exact match on the final numeric answer. Frontier models now achieve >95%, and GSM8K is approaching saturation, though it retains value for evaluating smaller models and studying RL-driven reasoning emergence.

2.8.2 MATH

The MATH benchmark Hendrycks et al. (2021c) consists of 12,500 competition-level mathematics problems across seven subject areas (Prealgebra, Algebra, Number Theory, Counting & Probability, Geometry, Intermediate Algebra, Precalculus) and five difficulty levels. Answers are in LaTeX `\boxed{}` format. At release, GPT-2 achieved only 6.9%; frontier reasoning models now exceed 95%.

2.8.3 MATH-500

MATH-500, introduced by Lightman et al. (2023), is a curated 500-problem subset of the MATH test set, representative across all subjects and difficulty levels. DeepSeek-R1-Zero achieved 95.9% on this subset. It is one of the primary evaluation benchmarks in this work.

2.8.4 AIME

The American Invitational Mathematics Examination consists of 15 problems with integer answers (0–999), representing a harder-tier benchmark. DeepSeek-R1 reported 79.8% pass@1 on AIME 2024.

2.8.5 Chain-of-Thought and Its Connection to RL-Trained Reasoning

Chain-of-thought (CoT) prompting Wei et al. (2022) dramatically improved LLM performance on mathematical benchmarks by eliciting intermediate reasoning steps. The key insight underlying DeepSeek-R1-Zero is that RL training can cause CoT-like behaviors to *emerge spontaneously*: the model discovers that generating intermediate reasoning steps is instrumentally useful for arriving at correct answers.

Table 2.4: Comparison of mathematical reasoning benchmarks.

Benchmark	Size	Difficulty	Answer Format	Frontier SOTA	Primar
GSM8K	1,319 (test)	Grade school	Numeric	∼97%	Baseline
MATH	5,000 (test)	Competition	LaTeX (`\boxed{}`)	>95%	Standard
MATH-500	500	Competition	LaTeX (`\boxed{}`)	∼99%	Efficient
AIME 2024	30	Olympiad-track	Integer (0–999)	∼96%	Frontier
Code Generation & Software Engineering					
HumanEval	164	Introductory	Pass/Fail (tests)	∼99%	Code sy
MBPP	974	Introductory	Pass/Fail (tests)	∼95%	Code sy
LiveCodeBench	600+	Competition	Pass/Fail (tests)	∼65%	Contami
SWE-bench Verified	500	Real-world	Patch resolution	∼72%	Software
Scientific & General Reasoning					
GPQA Diamond	198	Graduate	Multiple choice	∼92%	Expert-l
MMLU-Pro	12,000+	Undergraduate+	Multiple choice (10)	∼85%	General
OlympiadBench	8,476	Olympiad	Open-ended	∼60%	Multimo
ARC-AGI-2	400	Abstract	Grid transform	<10%	Fluid in
MMMU	11,500	Expert	Multiple choice	∼78%	Multimo
Additional Math & Code Datasets					
AMC 10/12	25 each	High school	Multiple choice (5)	∼95%	Mid-tier
Minerva Math	272	University	Open-ended	∼90%	STEM r
APPS	10,000	Mixed	Pass/Fail (tests)	∼80%	Algorith
CodeContests	13,328	Competition	Pass/Fail (tests)	∼45%	Competi
MathVista	6,141	Mixed	Mixed	∼70%	Visual m

2.8.6 Code Generation Benchmarks

HumanEval and MBPP

HumanEval Chen et al. (2021) is a benchmark of 164 hand-crafted Python programming problems introduced by OpenAI. Each problem includes a function signature, a docstring specification, and a suite of unit tests. The model must synthesize a correct function body from the docstring; evaluation uses the *pass@k* metric, measuring whether at least one of k generated samples passes all test cases. At release, Codex-12B achieved 28.8% pass@1; as of early 2026, frontier models exceed 99%.

The Mostly Basic Python Problems (MBPP) benchmark Austin et al. (2021) complements HumanEval with 974 crowd-sourced Python programming tasks targeting introductory-level skills. Each task provides a natural language description and three test cases. MBPP tests a broader distribution of basic programming patterns than HumanEval. Both benchmarks have approached saturation for frontier models, leading to the development of enhanced variants (HumanEval+ and MBPP+) with expanded test suites that expose edge-case failures Liu et al. (2024).

LiveCodeBench

LiveCodeBench Jain et al. (2025), published at ICLR 2025, addresses the critical problem of *data contamination* in code benchmarks. It continuously collects new competitive programming problems from LeetCode, AtCoder, and Codeforces after a given cutoff date, ensuring that problems were not present in any model's training data. Beyond code generation, LiveCodeBench evaluates self-repair, code execution prediction, and test output prediction capabilities. The benchmark currently hosts over 600 problems and has been instrumental in detecting contamination across models including GPT-4o, Claude, and DeepSeek. DeepCoder-14B, a GRPO+-trained model, achieved 60.6% on LiveCodeBench Xu et al. (2025a), demonstrating the applicability of RL-based training to code reasoning.

SWE-bench

SWE-bench Jimenez et al. (2024), published at ICLR 2024, evaluates whether language models can resolve real-world GitHub issues by generating code patches. The benchmark sources task instances from popular open-source Python repositories, each consisting of a GitHub issue description and a test suite that validates the fix. The full benchmark contains 2,294 instances; the *SWE-bench Verified* subset of 500 instances has been validated by human software engineers from OpenAI to ensure solvability and unambiguous specifications.

SWE-bench represents a qualitative shift from function-level code generation to repository-level software engineering—models must navigate multi-file codebases, understand project conventions, and produce patches that pass both new and regression tests. As of early 2026, the strongest agentic systems achieve approximately 72% on SWE-bench Verified. SWE-bench has become the de facto standard for evaluating AI coding agents, and its multimodal and multilingual extensions further broaden the evaluation scope.

2.8.7 Scientific and General Reasoning Benchmarks

GPQA Diamond

The Graduate-Level Google-Proof Q&A (GPQA) benchmark Rein et al. (2024), developed by researchers at NYU and Anthropic, evaluates expert-level scientific reasoning. The *Diamond* subset consists of 198 multiple-choice questions in biology, physics, and chemistry, crafted by PhD-level domain experts and validated through a rigorous multi-stage protocol. Questions are designed to be "Google-proof"—skilled non-experts with unrestricted internet access achieve only ∼34% accuracy, while domain experts score ∼65–70%.

GPQA Diamond has served as a critical benchmark for tracking the frontier of scientific reasoning in LLMs. At launch, GPT-4 achieved only ∼39%. By mid-2024, OpenAI's o1 reached ∼77%, surpassing human expert performance. As of early 2026, top models exceed 92%. The benchmark's relevance to RL-trained reasoning models is significant: DeepSeek-R1 achieved 71.5% on GPQA Diamond, demonstrating that GRPO-based training can enhance scientific reasoning beyond mathematics.

MMLU and MMLU-Pro

The Massive Multitask Language Understanding (MMLU) benchmark Hendrycks et al. (2021b) evaluates knowledge and reasoning across 57 diverse subjects spanning STEM, humanities, social sciences, and professional domains. Each subject contains multiple-choice questions at difficulty levels ranging from elementary to advanced professional. MMLU has been one of the most widely cited LLM benchmarks, though frontier models now routinely exceed 90%, reducing its discriminative power.

MMLU-Pro Wang et al. (2024) addresses these limitations by expanding answer choices from 4 to 10, eliminating trivially solvable questions, and focusing on reasoning-intensive tasks. The resulting benchmark of over 12,000 questions causes a 16–33% accuracy drop compared to original MMLU, restoring discriminative power. DeepSeek-R1 achieved 85.5% on MMLU, demonstrating that RL-trained reasoning capabilities transfer beyond mathematics to general knowledge domains.

OlympiadBench

OlympiadBench He et al. (2024), presented at ACL 2024, is a bilingual (English–Chinese) multimodal benchmark comprising 8,476 Olympiad-level problems in mathematics and physics. Unlike MATH or AIME, which are text-only, OlympiadBench includes problems with diagrams, graphs, and figures that require visual understanding. Problems are drawn from international science Olympiads and national competitions, with detailed step-by-step solutions provided for evaluation.

The benchmark is particularly relevant to the study of RL-trained reasoning models because it tests whether mathematical reasoning capabilities transfer to problems requiring both symbolic manipulation and visual interpretation. Frontier reasoning models achieve approximately 60% accuracy, indicating substantial room for improvement and preserving discriminative power for evaluating future RL-enhanced systems.

ARC-AGI

The Abstraction and Reasoning Corpus (ARC-AGI) Chollet (2019), introduced by François Chollet, evaluates *fluid intelligence*—the ability to solve novel problems without prior domain-specific knowledge. Each task presents a small number of input–

output grid pairs as demonstrations; the model must infer the underlying transformation rule and apply it to a held-out test grid. ARC-AGI is designed to resist memorisation: tasks require recognising abstract patterns (symmetry, counting, object manipulation) that are trivial for humans but remain extremely challenging for LLMs.

ARC-AGI-1 was the benchmark that pinpointed the moment in late 2024 when AI systems moved beyond pure memorisation, with OpenAI's o3-preview achieving a breakthrough score. ARC-AGI-2, released in March 2025, is substantially harder: pure LLMs score 0%, and public reasoning systems achieve only single-digit accuracy, while every task has been solved by at least two humans in under two attempts. ARC-AGI provides a complementary evaluation axis to mathematical benchmarks by testing general abstract reasoning rather than domain-specific knowledge.

MMMU

The Massive Multi-discipline Multimodal Understanding (MMMU) benchmark Yue et al. (2024) evaluates multimodal models on 11,500 expert-level questions spanning 30 subjects across six disciplines (Art & Design, Business, Science, Health & Medicine, Humanities, and Technology & Engineering). Each question requires understanding images—charts, diagrams, maps, chemical structures, medical scans, or artwork—alongside textual reasoning.

MMMU is particularly important for evaluating whether RL-trained reasoning capabilities extend to multimodal settings. At release, frontier models achieved approximately 56%; by early 2026, the best systems reach ∼78%, indicating that multimodal expert-level reasoning remains an active challenge. The benchmark has become the standard evaluation for vision-language models and is increasingly relevant as GRPO-style training is extended to multimodal architectures.

2.8.8 Additional Evaluation Datasets

AMC 10/12

The American Mathematics Competitions (AMC 10 and AMC 12) are widely-administered pre-AIME competitions consisting of 25 multiple-choice problems each, with 75-minute time limits. AMC problems are generally easier than AIME but harder than GSM8K, filling a useful difficulty gap. Problems test algebra, geometry, combinatorics, and number theory at the high school level. SimpleRL-Zoo Zeng et al. (2025) uses AMC as one of its standard evaluation benchmarks, reporting that GRPO-trained models show consistent improvements across the AMC difficulty range.

Minerva Math

The Minerva Math benchmark, curated from the evaluation suite of Google's Minerva model Lewkowycz et al. (2022), consists of quantitative reasoning problems drawn from STEM courses including calculus, linear algebra, probability, and physics. Problems require multi-step symbolic computation and domain knowledge beyond the competition-math style of MATH and AIME. Minerva Math has been adopted as a standard evaluation benchmark by several RL-for-reasoning works, including SimpleRL-Zoo and AceReason-Nemotron, providing coverage of university-level mathematical reasoning that complements the competition-oriented benchmarks.

APPS and CodeContests

APPS Hendrycks et al. (2021a) (Automated Programming Progress Standard) consists of 10,000 coding problems collected from open-access coding websites such as Codeforces and Kattis, spanning three difficulty levels: introductory, interview, and competition. Each problem includes multiple test cases for evaluation, and solutions require understanding of algorithms, data structures, and mathematical reasoning.

CodeContests Li et al. (2022), introduced by DeepMind, is a competitive programming benchmark drawn from Codeforces contests with additional problems from other platforms. It includes 13,328 problems with extensive test cases (both public and hidden). Both benchmarks evaluate a model's ability to translate natural language problem descriptions into correct programs, testing algorithmic reasoning—a capability closely related to the chain-of-thought reasoning that GRPO training aims to enhance.

MathVista

MathVista Lu et al. (2024) is a consolidated mathematical reasoning benchmark designed to evaluate models on tasks requiring *visual* mathematical understanding. It aggregates 6,141 problems from 28 existing datasets and 3 newly created datasets, spanning five primary mathematical reasoning types: algebraic reasoning, arithmetic reasoning, geometry reasoning, numeric common sense, and statistical reasoning. Problems include function plots, geometric diagrams, tables, bar charts, and scientific figures.

MathVista bridges the gap between text-only math benchmarks (GSM8K, MATH) and general multimodal benchmarks (MMMU), providing targeted evaluation of whether mathematical reasoning capabilities extend to visual inputs. At release, GPT-4V achieved 49.9%; as of early 2026, frontier multimodal reasoning models reach approximately 70%, indicating ongoing challenges in visual mathematical reasoning.

2.9 Related Work on RL for LLM Reasoning

The release of DeepSeek-R1 in January 2025 ignited a wave of research into RL as a mechanism for eliciting and scaling reasoning capabilities. This section surveys the most influential concurrent efforts.

2.9.1 DeepScaleR: Iterative Context Lengthening

DeepScaleR Luo et al. (2025), developed at UC Berkeley and Agentica, demonstrated that RL can be scaled effectively even with a *1.5B parameter* model. The central innovation is **iterative context lengthening**: training begins with shorter context windows (8K tokens) and progressively extends to 16K and 24K. DeepScaleR-1.5B achieved **43.1% Pass@1 on AIME 2024**, matching OpenAI's o1-preview. The training dataset comprised ∼40,000 math problems, and the entire recipe was open-sourced.

2.9.2 SimpleRL-Zoo: Systematic Investigation of Zero RL Training

SimpleRL-Zoo Zeng et al. (2025), published at COLM 2025, studied zero RL training across **10 diverse base models** spanning different families and scales. Key findings: (i) most prior reproduction efforts focused on Qwen2.5 models, which are unrepresentatively favorable starting points; (ii) the "aha moment" was observed for the first time in small models outside the Qwen family; (iii) base model quality is often more important than scale alone.

2.9.3 TinyZero: Minimal Reproduction of R1-Zero Reasoning

TinyZero Wen et al. (2025) reproduced the R1-Zero paradigm at the **3B parameter** scale on the Countdown number game for under $30 of compute. The 3B base model spontaneously developed self-verification, systematic search, and iterative revision—demonstrating the minimal conditions under which RL can induce genuine reasoning.

2.9.4 DeepCoder: Extending RL Reasoning to Code Generation

DeepCoder Xu et al. (2025b) extended the RL scaling paradigm to competitive programming, achieving **60.6% Pass@1 on LiveCodeBench** with a 14B model using GRPO+ (enhanced GRPO with DAPO-inspired modifications). The approach adopted iterative context lengthening and sparse binary rewards from test case execution.

2.9.5 Open-Reasoner-Zero and Other Reproductions

Open-Reasoner-Zero demonstrated that vanilla PPO with rule-based rewards—without KL regularization—achieved superior performance to DeepSeek-R1-Zero-Qwen-32B while requiring only 1/10 of the training steps. Other efforts include Dr. GRPO (correcting GRPO's length bias), Sky-T1 (achieving o1-preview performance with minimal resources), and Logic-RL (extending RL training to logical reasoning).

2.9.6 Scaling Trends in RL Post-Training

Tan et al. conducted a comprehensive investigation across the Qwen2.5 dense model series (0.5B–72B), establishing that: (1) under fixed compute, larger models trained for fewer RL steps outperform smaller models trained longer; (2) RL post-training follows predictable power-law scaling; (3) data reuse is effective, as final RL performance is governed by total optimization steps rather than sample uniqueness.

2.10 Infrastructure for RL Training of LLMs

RL training introduces infrastructure demands beyond supervised fine-tuning: on-policy sampling, reward scoring, and gradient computation must share or synchronize model weights. This section surveys the key tools.

2.10.1 TRL: Transformer Reinforcement Learning

TRL von Werra et al. (2022) provides a unified interface for post-training with PPO, DPO, GRPO, and more. Its `GRPOTrainer` class manages the full RL loop and supports custom reward functions, with distributed training via DeepSpeed and FSDP.

2.10.2 OpenRLHF

OpenRLHF is a high-performance distributed RLHF framework combining Ray, vLLM, and DeepSpeed. It supports PPO, REINFORCE++, GRPO, RLOO, and DAPO algorithms, with tight vLLM integration for high-throughput on-policy generation.

2.10.3 DeepSpeed and FSDP

DeepSpeed's ZeRO optimizer **?** partitions optimizer states (Stage 1), gradients (Stage 2), and parameters (Stage 3) across data-parallel ranks, reducing per-GPU memory consumption to $O(1/N)$. PyTorch FSDP provides a native alternative. Both are critical enablers for RL training where multiple large models must coexist in memory.

2.10.4 vLLM

vLLM **?** addresses the inference bottleneck through PagedAttention and continuous batching, achieving 10–24× higher throughput than naive inference. In RL training, generation can account for up to 90% of wall-clock time, making vLLM essential for practical GRPO training.

2.10.5 Tinker: Training-as-a-Service

Tinker, developed by Thinking Machines Lab, exposes a cloud-based Python API with four primitive operations: `forward_backward`, `optim_step`, `sample`, and `save_state`. Researchers write training loops locally on CPU while Tinker orchestrates distributed GPU computation. The platform supports models from Qwen3.5-4B to Qwen3.5-397B-A17B via LoRA fine-tuning, significantly democratizing access to RL research.

2.10.6 Atropos and the Tinker–Atropos Integration

Atropos, developed by Nous Research, is an environment microservice framework that decouples environments, trajectory batching, and training into independently deployable components. The *tinker-atropos* integration connects Atropos environments to Tinker's managed compute, enabling any Atropos environment to leverage Tinker's GPU infrastructure. Experimental results on GSM8k demonstrate comparable reward convergence with lower per-step training time due to asynchronous decoupling of inference and gradient computation.

2.11 Scaling Laws and Empirical Observations

2.11.1 Classical Scaling Laws for Pretraining

Kaplan et al. established that test loss obeys smooth power-law relationships **?**:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \qquad L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}, \qquad L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}, \tag{2.19}$$

where N denotes parameters, D training tokens, C compute, with exponents $\alpha_N \approx 0.076$, $\alpha_D \approx 0.095$, $\alpha_C \approx 0.050$.

2.11.2 Compute-Optimal Training: Chinchilla Scaling

Hoffmann et al. showed that model size and training tokens should be scaled equally **?**:

$$L(N, D) = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + E, \tag{2.20}$$

with $\alpha \approx 0.34$, $\beta \approx 0.28$. Under fixed compute $C \propto 6ND$, optimal allocation scales both N and D proportionally. Chinchilla (70B, 4× more data) uniformly outperformed Gopher (280B).

2.11.3 Scaling Laws for Reinforcement Learning Fine-Tuning

RL training exhibits *sigmoidal* rather than power-law performance curves. Khatri et al. proposed:

$$R_C - R_0 = (A - R_0) \cdot \frac{1}{1 + (C_{\text{mid}}/C)^B}, \tag{2.21}$$

where A is the asymptotic pass rate and B governs compute efficiency. Key findings: (1) not all RL recipes reach the same ceiling; (2) methods superior at small compute can be inferior at scale; (3) scaling model size substantially improves both asymptotic performance and downstream generalization.

The effect of *group size* G is a scaling dimension unique to GRPO. DeepSeek-R1 used $G = 64$; academic reproductions find $G \in [8, 32]$ offers a practical trade-off.

2.11.4 Test-Time Compute Scaling

Snell et al. showed that compute-optimal test-time scaling—adaptively allocating inference compute per prompt—can achieve >4× improvement over uniform best-of-N baselines Snell et al. (2024). A smaller model with optimal test-time scaling outperformed a 14× larger model in zero-shot mode. This connects directly to R1-style long chain-of-thought reasoning, where RL teaches models to allocate their own test-time compute.

2.11.5 Reward Hacking and Overoptimisation

Gao et al. characterised reward model overoptimisation Gao et al. (2023): gold reward initially increases with KL divergence from the reference policy but eventually peaks and declines. Larger proxy reward models delay overoptimisation onset but do not eliminate it. For GRPO with rule-based rewards, overoptimisation can still manifest through formatting tricks or overfitting to training prompt distributions.

2.11.6 Implications for the Present Study

This capstone focuses on the effect of base model size on GRPO training dynamics, holding the algorithm (GRPO), adaptation method (LoRA), training framework (TRL von Werra et al. (2022)), reward structure (rule-based verification), and benchmarks (GSM8K, MATH) constant. This design isolates what the scaling literature identifies as one of the most consequential variables, aiming to empirically locate the transition

point below which distillation dominates direct RL, and to provide actionable guidance for practitioners under resource constraints.

2.12 Summary

This chapter has traced the intellectual lineage of reinforcement learning for large language models, from its earliest formulations through to the latest open-source reproductions.

The foundational paradigm of **RLHF with PPO** established that human preference signals could steer pretrained language models toward helpful behaviour Ouyang et al. (2022), but carried substantial engineering complexity: a four-model orchestration with extensive hyperparameter tuning Schulman et al. (2017); Zheng et al. (2023).

Direct Preference Optimization (DPO) offered an elegant shortcut by collapsing the RLHF pipeline into supervised learning on preference pairs Rafailov et al. (2023), yet introduced dependence on static data and the absence of online exploration.

Group Relative Policy Optimization (GRPO) Shao et al. (2024) charted a middle path, replacing learned value estimation with group-relative advantage normalization. The resulting algorithm retained PPO's clipped surrogate objective and KL regularisation while halving the number of trainable models.

DeepSeek-R1 DeepSeek-AI (2025) demonstrated that pure GRPO training with rule-based rewards could elicit emergent reasoning—self-verification, reflection, and extended chains of thought—without curated human demonstrations.

Parameter-efficient fine-tuning via LoRA Hu et al. (2022) and QLoRA Dettmers et al. (2023) has been indispensable in democratising access to RL training, making GRPO feasible on academic-grade hardware.

Evaluation benchmarks spanning mathematical reasoning (GSM8K, MATH, AIME, AMC, Minerva Math, OlympiadBench, MathVista), code generation (HumanEval, LiveCodeBench, SWE-bench, APPS, CodeContests), and scientific/general reasoning (GPQA Diamond, MMLU-Pro, MMMU, ARC-AGI) collectively provide verifiable ground-truth answers enabling rule-based reward computation and comprehensive capability assessment.

A **growing ecosystem of open-source reproductions**—TinyZero Wen et al. (2025), SimpleRL-Zoo Zeng et al. (2025), DeepScaleR Luo et al. (2025), and DeepCoder Xu et al. (2025b)—has validated that GRPO-based training can be replicated at various scales using open data and commodity hardware.

Positioning of this study. The present capstone project contributes to this landscape by conducting a controlled, systematic investigation of how GRPO training dynamics, emergent reasoning behaviours, and downstream mathematical performance

scale across model sizes. By holding the algorithm (GRPO), the adaptation method (LoRA), the training framework (TRL von Werra et al. (2022)), and the evaluation benchmarks constant while varying model capacity, this study isolates the effect of scale on reinforcement-learned reasoning—a dimension that existing work has explored only incidentally.

Bibliography

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized language models. *NeurIPS*, 2023.

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. *ICML*, 2023.

Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, et al. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In *ACL*, 2024.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. APPS: Measuring coding challenge competence with APPS. *NeurIPS*, 2021a.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *ICLR*, 2021b.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *NeurIPS*, 2021c.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida I. Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *ICLR*, 2025.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *NeurIPS*, 2022.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. *NeurIPS*, 2024.

Pan Lu, Hritik Banber, Botao Ran, Lili Liang, Zhiyong Lin, Jiaheng Qiu, Ruochen Ren, Yan Gao, Guoli Pei, Chenyang Zhang, et al. MathVista: Evaluating mathematical reasoning of foundation models in visual contexts. *ICLR*, 2024.

Michael Luo et al. DeepScaleR: Surpassing O1-Preview with a 1.5B model by scaling RL. *arXiv preprint*, 2025.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *NeurIPS*, 2023.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level Google-Proof Q&A benchmark. *arXiv preprint arXiv:2311.12022*, 2024.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, and Shengyi Huang. TRL: Transformer reinforcement learning. *GitHub repository*, 2022.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *NeurIPS*, 2024.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 2022.

Jiayi Wen et al. TinyZero: Reproducing DeepSeek-R1-Zero in countdown and beyond. *arXiv preprint*, 2025.

Xiang Xu et al. DeepCoder: A fully open-source 14B coder at O3-mini level. *arXiv preprint*, 2025a.

Xiang Xu et al. DeepCoder: A fully open-source 14B coder at O3-mini level. *arXiv preprint*, 2025b.

Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. MMMU: A massive multi-discipline multimodal understanding and reasoning benchmark for expert AGI. *CVPR*, 2024.

Weihao Zeng et al. SimpleRL-Zoo: Investigating and taming zero reinforcement learning for open base models. *arXiv preprint*, 2025.

Rui Zheng, Shihan Dou, Songyang Gao, Yuan Hua, Wei Shen, Binghai Wang, Yan Liu, Senjie Jin, Qin Liu, Yuhao Zhou, et al. Secrets of RLHF in large language models part I: PPO. *arXiv preprint arXiv:2307.04964*, 2023.